

Trait — TOÀN BỘ qua HÌNH VẼ

Companion visual cho [trait.md](#). Mọi khái niệm được vẽ ra. Đọc tuần tự — mỗi hình xây trên hình trước.

Mục lục

1. Bức tranh lớn: Trait là gì?
2. Trait vs Class — so sánh tư duy
3. Khai báo & implement
4. Default method
5. Supertrait
6. Method vs Associated Function
7. Static dispatch — Monomorphization
8. Dynamic dispatch — vtable
9. Fat pointer của trait object
10. Layout vtable
11. So sánh Static vs Dynamic
12. Object safety — rule trực quan
13. Associated type
14. Generic parameter trên trait
15. Marker trait — Send/Sync flow
16. Orphan rule — coherence
17. Newtype pattern
18. Blanket implementation
19. Iterator — chain & lazy
20. Deref coercion chain
21. From → Into auto
22. Bản đồ trait stdlib
23. Decision tree: static hay dynamic?
24. Bản đồ tư duy tổng

1. Bức tranh lớn

```

    TRAIT
= "Khả năng" mà một TYPE có thể có
Hợp đồng:
  "Type X tuyên bố có khả năng Y"
Code:
  trait Y {
```

```

    fn method1(&self) -> ...;
}

impl Y for X {
    fn method1(&self) -> ... { ... }
}

```

Ví dụ thực:

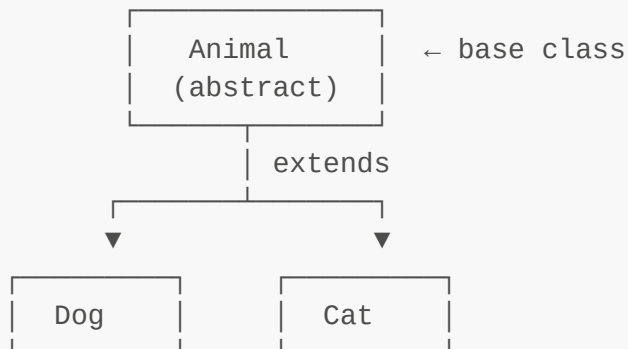
```

trait Bark { fn bark(&self); }
    |
    | "khả năng sủa"
    |
    | → impl Bark for Dog { ... }      ✓ Dog có khả năng sủa
    | → impl Bark for Wolf { ... }    ✓ Wolf có khả năng sủa
    | → impl Bark for Cat { ... }     ✗ không có (sai semantic)

```

2. Trait vs Class

JAVA / C++ (OOP)



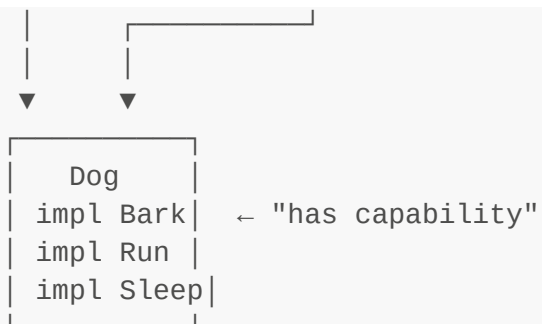
Quan hệ: IS-A (kế thừa cứng)
 Field + method từ parent → child
 "Dog IS-A Animal"

RUST (Trait)

```

trait Bark
trait Run
trait Sleep
    |
    | CAPABILITIES
    |
    | struct Dog
    | { name, ... }

```



Quan hệ: HAS-CAPABILITY (composition + trait)
Field tự định nghĩa, capability mix-and-match

3. Khai báo & implement

```

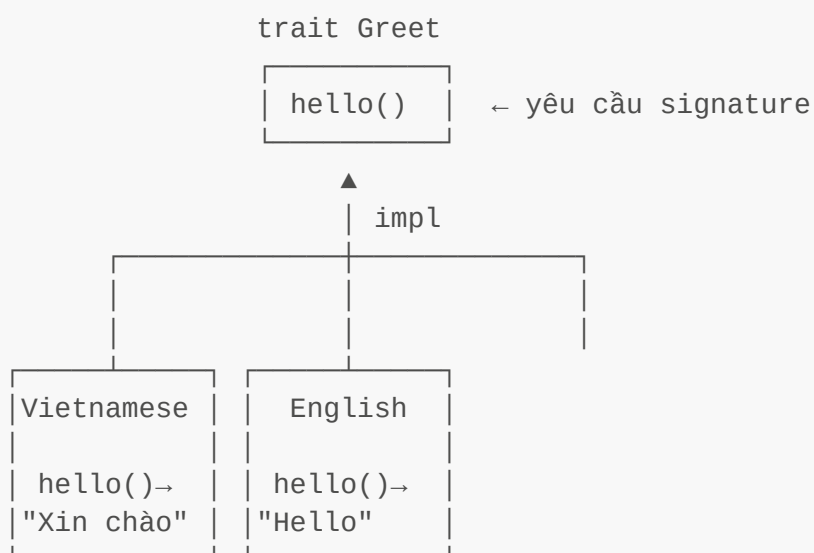
trait Greet {
    fn hello(&self) -> String;
}

struct Vietnamese;
struct English;

impl Greet for Vietnamese {
    fn hello(&self) -> String { String::from("Xin chào") }
}

impl Greet for English {
    fn hello(&self) -> String { String::from("Hello") }
}
    
```

Sơ đồ



Mỗi struct ĐỘC LẬP impl trait theo cách riêng.

Gọi method

```
let v = Vietnamese;
println!("{}", v.hello());    // → "Xin chào"

let e = English;
println!("{}", e.hello());    // → "Hello"
```

compile time



Compiler biết v: Vietnamese → direct call (static)

4. Default method

```
trait Print {
    fn name(&self) -> String;           // required

    fn announce(&self) {                // default
        println!("Tôi là {}", self.name());
    }
}

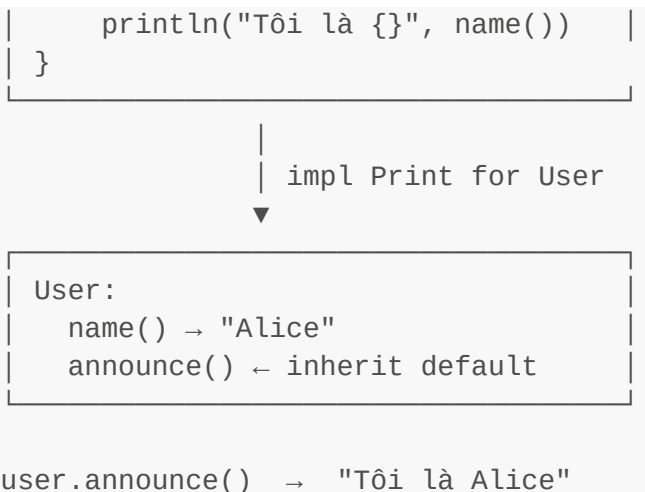
struct User { name: String }

impl Print for User {
    fn name(&self) -> String { self.name.clone() }
    // KHÔNG impl announce → dùng default
}
```

Sơ đồ

trait Print:





Override

```

impl Print for VIP {
  fn name(&self) -> String { ... }

  fn announce(&self) { // override
    println!("✨ {} ✨", self.name());
  }
}
  
```

trait Print default

announce: print "Tôi là X"

VIP override

announce: print "✨ X ✨"

5. Supertrait

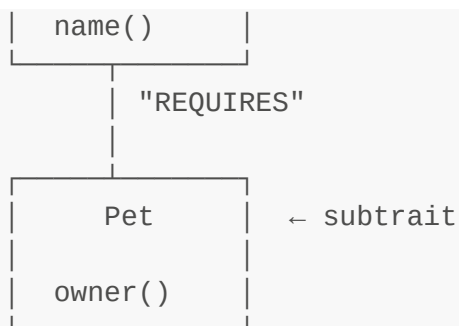
```

trait Animal {
  fn name(&self) -> String;
}

trait Pet: Animal { // Pet REQUIRES Animal
  fn owner(&self) -> String;
}
  
```

Sơ đồ phân cấp





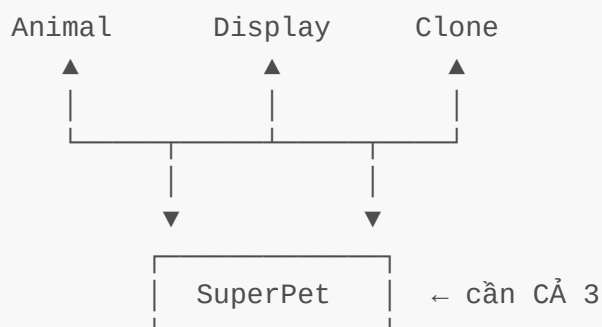
Type nào impl Pet → phải impl Animal trước:

```
impl Animal for Dog { fn name() {...} } ✓ required
impl Pet      for Dog { fn owner() {...} } ✓ enabled
```

Thiếu Animal → ERROR khi impl Pet

Nhiều supertrait

```
trait SuperPet: Animal + Display + Clone {
  fn intro(&self) {
    // dùng được name() (từ Animal), Display ({}), clone()
  }
}
```



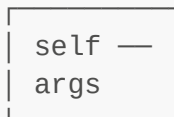
6. Method vs Associated Function

```
trait Shape {
  fn area(&self) -> f64;           // METHOD (có self)
  fn unit() -> Self;              // ASSOCIATED FN (no self)
}
```

Sơ đồ gọi

Method:

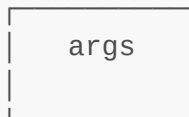
instance.method(args)



```
c = Circle { ... }
a = c.area()
```

Associated fn:

Type::function(args)



```
c = Circle::unit()
(constructor)
```

7. Static dispatch — Monomorphization

Code bạn viết

```
fn print<T: Greet>(x: &T) {
    println!("{}", x.hello());
}

let v = Vietnamese;
let e = English;
print(&v);
print(&e);
```

Compiler sinh ra

MONOMORPHIZATION (compile time)

Phát hiện: print được gọi với T = Vietnamese, English

→ CLONE function thành 2 bản:

```
fn print__Vietnamese(x: &Vietnamese) {
    println!("{}", x.hello());    ← inline Vietnamese::hello
}
```

```
fn print__English(x: &English) {
    println!("{}", x.hello());    ← inline English::hello
}
```

Caller code đổi thành:

```
print__Vietnamese(&v);
```

```
print__English(&e);
```

Sơ đồ binary

BEFORE monomorphization:

```
fn print<T>(x: &T)
    |
    | (generic,
    |   chưa thực sự ở
    |   binary)
    |
    ▼
1 placeholder
```

AFTER:

```
fn print__Vietnamese(x: &V)
fn print__English(x: &E)
fn print__Dog(x: &D)
...
(nhiều function, mỗi cái
optimize cho type cụ thể)

N functions trong binary
```

Cost: code size tăng

Code size:

Generic 1000-line × 10 types = 10,000 lines code thật trong binary

↑ tăng binary size

↓ giảm icache hit nếu binary quá lớn

Trade-off: speed ↑↑↑ vs size ↑↑

8. Dynamic dispatch — vtable

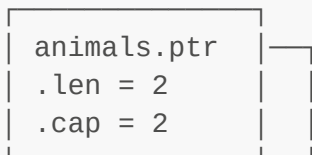
Code

```
let animals: Vec<Box<dyn Greet>> = vec![
    Box::new(Vietnamese),
    Box::new(English),
];

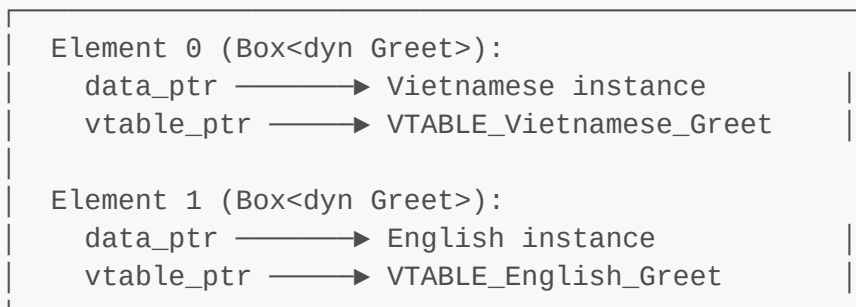
for a in &animals {
    println!("{}", a.hello());
}
```

Sơ đồ memory

STACK:



HEAP (Vec backing):



TEXT segment (static, in binary):



Diễn biến **a.hello()**

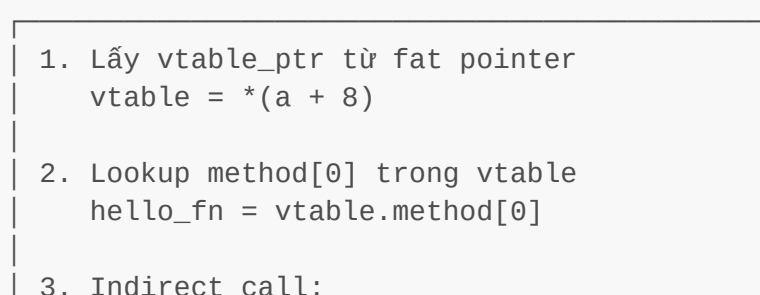
```
for a in &animals:
```



```
a = &Box<dyn Greet>
```



```
a.hello():
```

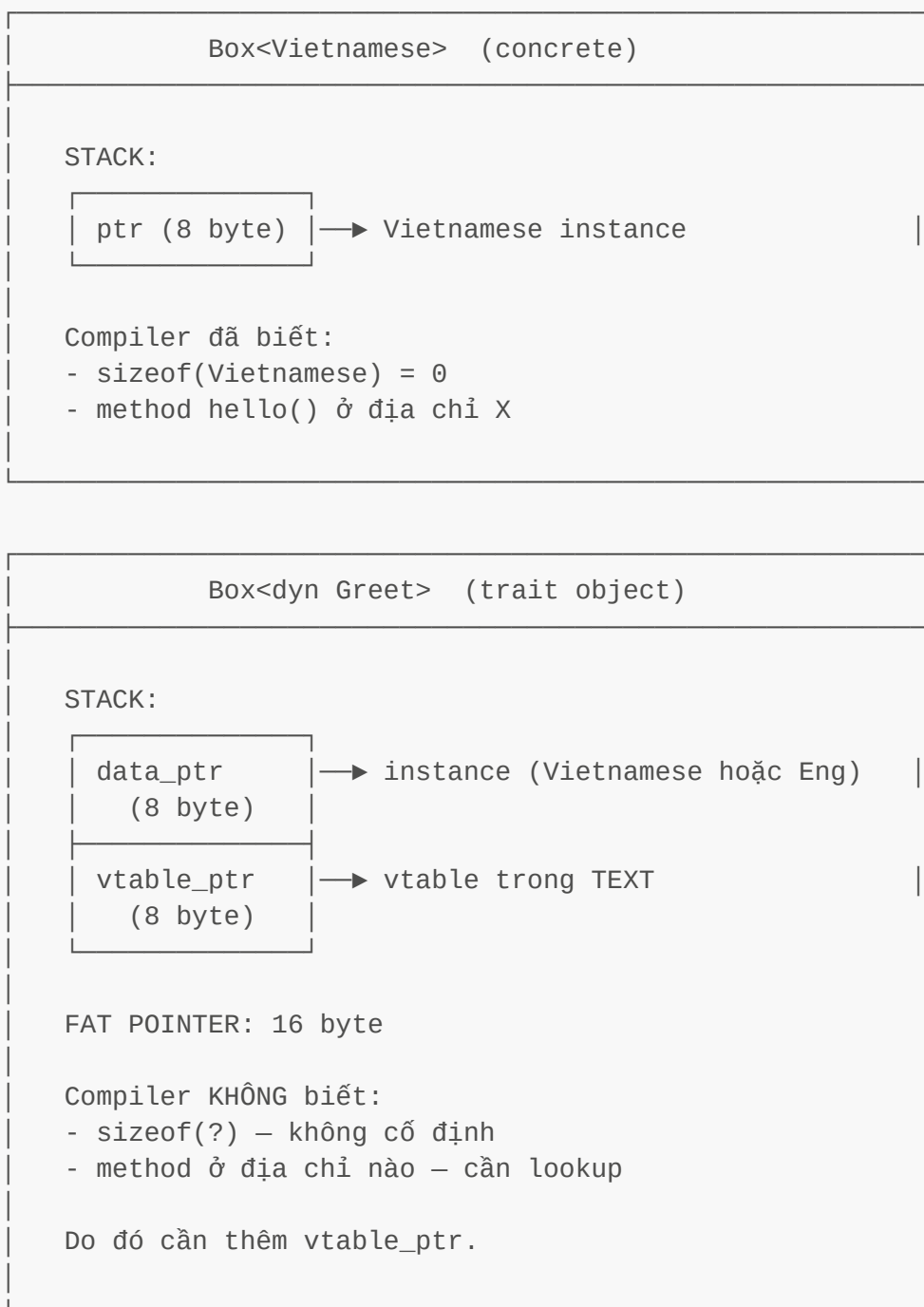


```
hello_fn(data_ptr)
```

- 1 memory load + 1 indirect call
- ~2-3 ns extra cost

9. Fat pointer

So sánh pointer thường vs trait object



Vì sao 2 pointer?

→ Cả 2 ptr → 16 byte → "fat" pointer

drop_dog size: 24 align: 8	drop_cat size: 16 align: 8
Dog::name Dog::sound Dog::age	Cat::name Cat::sound Cat::age

STATIC trong binary, mỗi (Type, Trait) có 1 vtable duy nhất.

11. So sánh Static vs Dynamic

STATIC DISPATCH

```
fn process<T: Greet>(x: &T)
```

monomorphize

Compile → multiple copies:

process	process	process	...
__Viet	__Eng	__Dog	

⚡ Speed: ⚡⚡⚡ (inline, optimized per type)

📦 Binary: ↑↑ (nhiều bản code)

🧠 Compile: 🐢 (slow)

DYNAMIC DISPATCH

```
fn process(x: &dyn Greet)
```

trait object

Compile → 1 copy:

process	← vtable lookup mỗi call
---------	--------------------------

⚡ Speed: ⚡⚡ (~2-3 ns extra per call)

📦 Binary: ↓ (chỉ 1 bản code)

🧠 Compile: ⚡ (fast)

Lookup chain:

```

call x.hello()
|
▼ load vtable_ptr
|
▼ index method
|
▼ indirect call
|
▼ execute

```

Benchmark visual

100,000,000 calls trong loop:

Static dispatch:  0.3s

Dynamic dispatch:  0.6s

→ Khác biệt ~2x cho hot loop

→ Cho app thường: không cảm nhận được

12. Object safety — rule trực quan

OBJECT-SAFE RULES

✓ ĐƯỢC làm dyn:

```

trait Good {
  fn m(&self);
  fn m2(&self) -> i32;
}

trait WithDefault {
  fn m(&self) {...}
}

```

✗ KHÔNG được:

```

trait Bad1 {
  fn m<T>(&self, x: T);
} // ↑ generic

trait Bad2 {
  fn m(&self) -> Self;
} // ↑ return Self

trait Bad3 {
  fn helper() -> i32;
} // ↑ no self / static

```

Quy tắc visual

Mỗi method trong trait phải:

```
[ ] Có &self / &mut self / self / Box<self>
    (NOT static)

[ ] Không có generic <T> trên method
    (Lifetime OK, type generic NO)

[ ] Return NOT Self
    (trừ khi dùng &Self / &mut Self / Box<Self>)

[ ] Không where Self: Sized

Nếu MỌI method thoả → trait là OBJECT-SAFE
→ có thể dùng dyn Trait
```

Vì sao? Trực quan

Generic method:

```
trait Bad { fn m<T>(&self, x: T); }
```

vtable cần lưu m::<i32>, m::<f64>, m::<String>, ...

Vô hạn instantiation → KHÔNG ĐỦ chỗ trong vtable

→ CẤM

Return Self:

```
trait Bad { fn clone(&self) -> Self; }
```

Caller có &dyn Trait, không biết Self là gì.

→ KHÔNG biết stack alloc bao nhiêu cho return value

→ CẤM

Static fn:

```
trait Bad { fn unit() -> Self; }
```

Gọi qua dyn nào? `(dyn Trait)::unit()` – không có instance

→ KHÔNG hợp lý

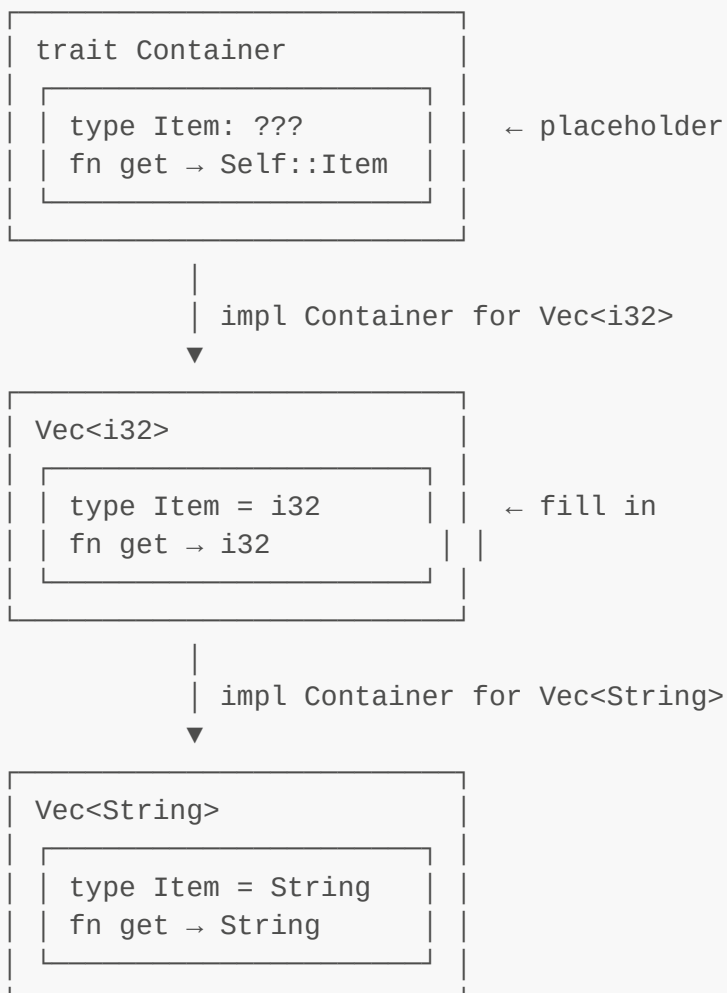
→ CẤM (trừ khi có where Self: Sized)

13. Associated type

```
trait Container {
    type Item;
```

```
fn get(&self, i: usize) -> Self::Item;
}
```

Sơ đồ



Mỗi type chỉ impl Container 1 LẦN.
Item là HỆ QUẢ của type, không phải chọn lúc gọi.

14. Generic parameter trên trait

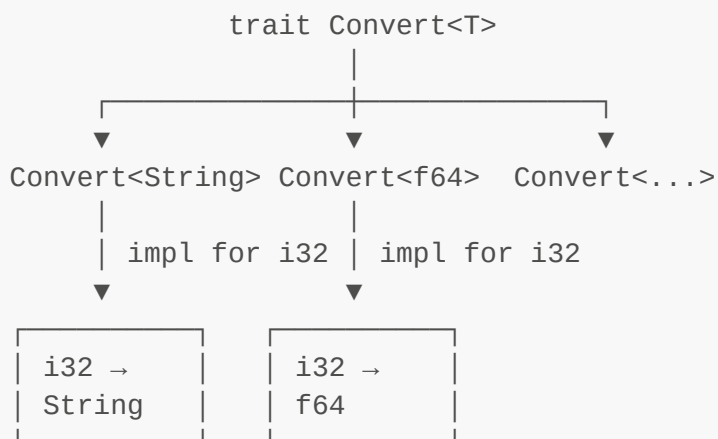
```
trait Convert<T> {
    fn convert(self) -> T;
}

impl Convert<String> for i32 {
    fn convert(self) -> String { self.to_string() }
}

impl Convert<f64> for i32 {
```

```
fn convert(self) -> f64 { self as f64 }
}
```

Sơ đồ



i32 có NHIỀU impl Convert với T khác nhau.
Mỗi T = 1 impl khác nhau.

So sánh

ASSOCIATED TYPE

type Item

1 type → 1 impl
Output PHỤ THUỘC type

```
Iterator
trait Iterator {
    type Item;
}
```

Vec<i32>::iter() → Item=&i32
(chỉ 1 cách)

GENERIC PARAMETER

<T>

1 type → N impl
Output CHỌN lúc gọi

```
From
trait From<T> {
    fn from(t: T) -> Self;
}
```

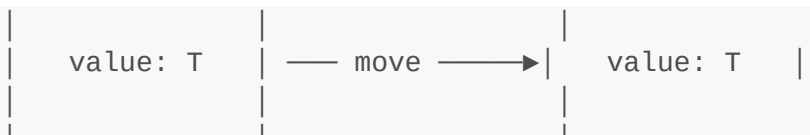
i32: From<u8>, From<i8>, ...
(nhiều)

15. Send/Sync flow

Send: chuyển ownership giữa thread

Thread A

Thread B



Hợp pháp KHI: T: Send

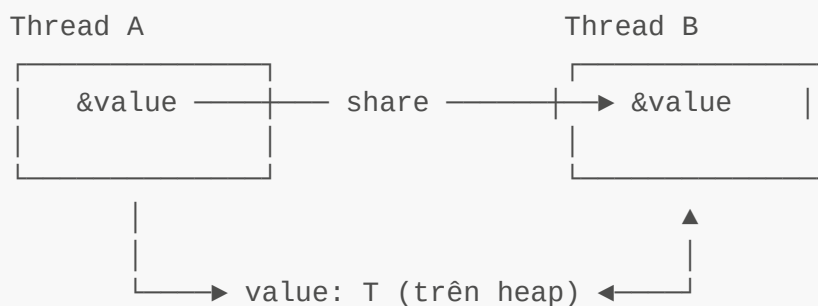
T: Send

i32, String, Vec
Box<T> if T: Send
Arc<U> if U: Send + Sync

T: NOT Send

Rc<U> (count không atomic)
*mut U (raw ptr)
MutexGuard (lock thread-specific)

Sync: chia sẻ &T giữa thread



Hợp pháp KHI: T: Sync

T: Sync

i32, String
atomic)
Mutex<U>
AtomicI32
&T if T: Sync

T: NOT Sync

Cell<U> (interior mut, không
RefCell<U> (counter không atomic)
Rc<U>

Mối liên hệ Send/Sync

T: Sync $\Leftrightarrow$ &T: Send

Đọc:

"T có thể chia sẻ giữa thread"

tương đương

"&T có thể chuyển giữa thread"

Auto-derivation

```
struct MyData {
    a: i32,          ← Send + Sync
    b: String,       ← Send + Sync
    c: Vec<i32>,     ← Send + Sync
}
```

→ MyData TỰ ĐỘNG: Send + Sync

```
struct MyDataRc {
    a: i32,
    b: Rc<i32>,      ← NOT Send, NOT Sync
}
```

→ MyDataRc TỰ ĐỘNG: NOT Send, NOT Sync
(1 field xấu → cả struct xấu)

16. Orphan rule

QUY TẮC: Để impl Trait for Type, ÍT NHẤT MỘT phải LOCAL

Trait local?	Type local?	OK?
✓	✓	✓
✓	✗	✓
✗	✓	✓
✗	✗	✗ ORPHAN!

Sơ đồ

YOUR CRATE

```
MyStruct
MyTrait
```

- ✓ impl MyTrait for MyStruct
- ✓ impl MyTrait for Vec<i32>
- ✓ impl Display for MyStruct
- ✗ impl Display for Vec<i32>

EXTERNAL CRATES

```
Vec, String
Display
```

- (cả 2 local)
- (trait local)
- (type local)
- (NEITHER local!) ← orphan

Vì sao cần rule này?

Giả sử không có orphan rule:

Crate A:

```
impl Display
  for Vec<i32>
  → "[1,2,3]"
```

Crate B:

```
impl Display
  for Vec<i32>
  → "[1, 2, 3]"
```

User crate:

```
use A;
use B;
vec.fmt(...) ? ← AMBIGUOUS, impl nào?
```

→ Đổ vỡ ecosystem. CẤM ngay từ đầu.

17. Newtype pattern

```
struct MyVec(Vec<i32>);           // wrap

impl Display for MyVec {         // ✓ MyVec local!
  fn fmt(&self, f: &mut Formatter) -> fmt::Result {
    write!(f, "...")
  }
}
```

Sơ đồ

Vấn đề:

```
impl Display for Vec<i32>
❌ orphan
```

Giải pháp:

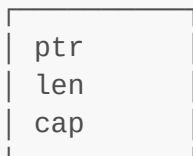
```
struct MyVec(Vec<i32>);
|
| wrap
▼
impl Display for MyVec ✓

let v = MyVec(vec![1,2,3]);
println!("{}", v);
```

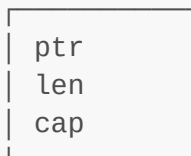
Layout: zero-cost wrapper

Vec<i32>:

STACK (24 byte):

MyVec(Vec<i32>):

STACK (24 byte):



↑ y hệt Vec! Không thêm gì.

→ Newtype = **zero runtime cost**.

18. Blanket implementation

```

impl<T: Display> ToString for T {
    fn to_string(&self) -> String {
        format!("{}", self)
    }
}

```

Sơ đồ phổ quát

Universal quantifier:

∀ T mà T: Display:

T tự động có ToString

Tức là:

```

i32: Display      → i32: ToString
f64: Display      → f64: ToString
String: Display   → String: ToString
MyType: Display   → MyType: ToString   (tự động!)
...

```

Không cần impl thủ công cho từng type.

Ứng dụng

```
// Bạn impl Display:
impl Display for User { ... }

// Tự dùng có:
let u = User { ... };
let s: String = u.to_string(); // ← blanket impl cho miễn phí
```

19. Iterator — chain & lazy

```
let v = vec![1, 2, 3, 4, 5];
let r: Vec<i32> = v.iter()
    .map(|x| x * 2)
    .filter(|x| x > &4)
    .collect();
```

Sơ đồ lazy

ITER STATE MACHINE:

```
v.iter()      → Iter (lazy, không tính gì)
  |
  ▼
.map(|x| x*2) → Map<Iter, F> (vẫn lazy)
  |
  ▼
.filter(...) → Filter<Map<Iter, F>, P> (vẫn lazy)
  |
  ▼
.collect()   → TRIGGER thực hiện!
```

Pipeline mỗi item:

```
1 → map → 2 → filter (2 > 4? NO) → discard
2 → map → 4 → filter (4 > 4? NO) → discard
3 → map → 6 → filter (6 > 4? YES) → push to Vec
4 → map → 8 → filter (8 > 4? YES) → push to Vec
5 → map → 10 → filter (10 > 4? YES) → push to Vec
```

```
r = [6, 8, 10]
```

Quan trọng: KHÔNG có Vec trung gian

Naive (Java/Python style):

```
tmp1 = v.map(...)
tmp2 = tmp1.filter(...)
result = tmp2.collect()
```

Memory: 3 Vec trung gian

Rust idiom:

Pipeline mỗi item
qua chain rồi mới đến item kế

Memory: chỉ result Vec



Zero-cost

Code Rust:

```
(0..1000)
  .filter(|x| x%2==0)
  .sum()
```

Sau optimize:
y hệt code C bên phải

Code C tương đương:

```
int sum = 0;
for (int i = 0; i < 1000; i++) {
    if (i%2 == 0) sum += i;
}
```

Tốc độ:
Y HẾT NHAU (~ns khác biệt)

20. Deref coercion chain

```
let s: String = "hi".to_string();
fn need_str(x: &str) {}
need_str(&s);           // &String → &str
```

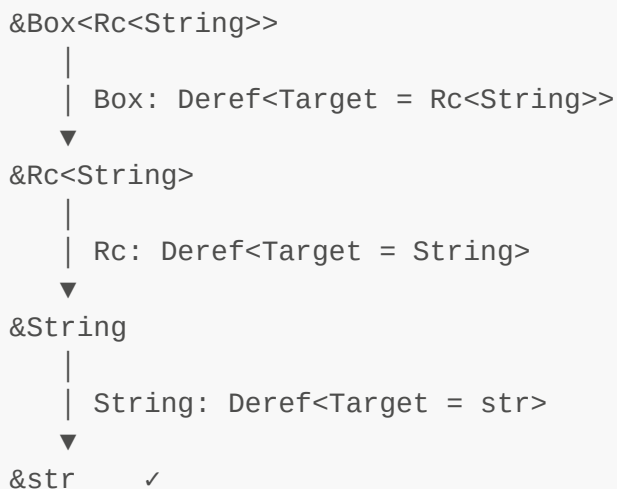
Sơ đồ chain

Pass &String đến hàm cần &str:

```
&String
  |
  | String: Deref<Target = str>
  ▼
&str    ✓ match!
```

Nhiều cấp

```
let b: Box<Rc<String>> = Box::new(Rc::new("hi".to_string()));
need_str(&b);
```



Compiler thử mọi cấp đến khi khớp.

Cảnh báo: chỉ cho smart pointer

✓ Đúng dùng:

```
impl Deref for Box<T> { ... }
impl Deref for Rc<T> { ... }
impl Deref for Arc<T> { ... }
impl Deref for MutexGuard<T> { ... }
```

✗ Sai dùng:

```
impl Deref for User { type Target = String; ... }
```

→ confuse: "user.len() là String::len? Hay User method?"

21. From → Into auto

```
impl From<i32> for MyNum {
    fn from(x: i32) -> MyNum { ... }
}
```

// TỰ ĐỘNG CÓ:

```
// impl Into<MyNum> for i32 { ... }
```

Cơ chế

trong std:

```
impl<T, U: From<T>> Into<U> for T {
  fn into(self) -> U {
    U::from(self)
  }
}
```

↑ Blanket impl

Sơ đồ

Bạn impl:

```
impl From<i32> for MyNum
```

| (chỉ cần 1 chiều)

▼
MyNum::from(5)

| cùng kết quả
| MyNum

Auto sinh:

```
impl Into<MyNum> for i32
```

| (chiều ngược tự có)

▼
5.into()

|

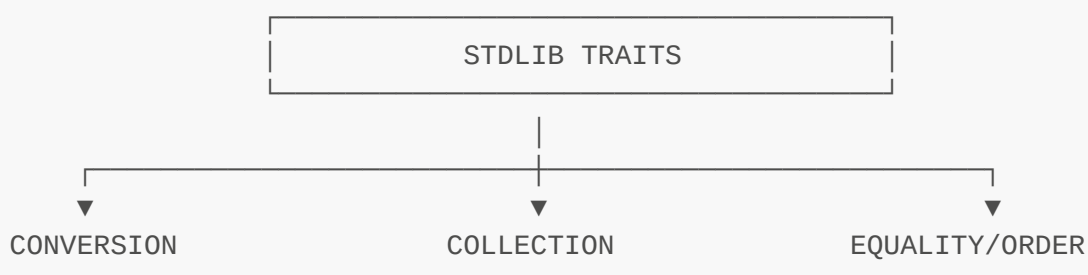
Quy tắc đẹp

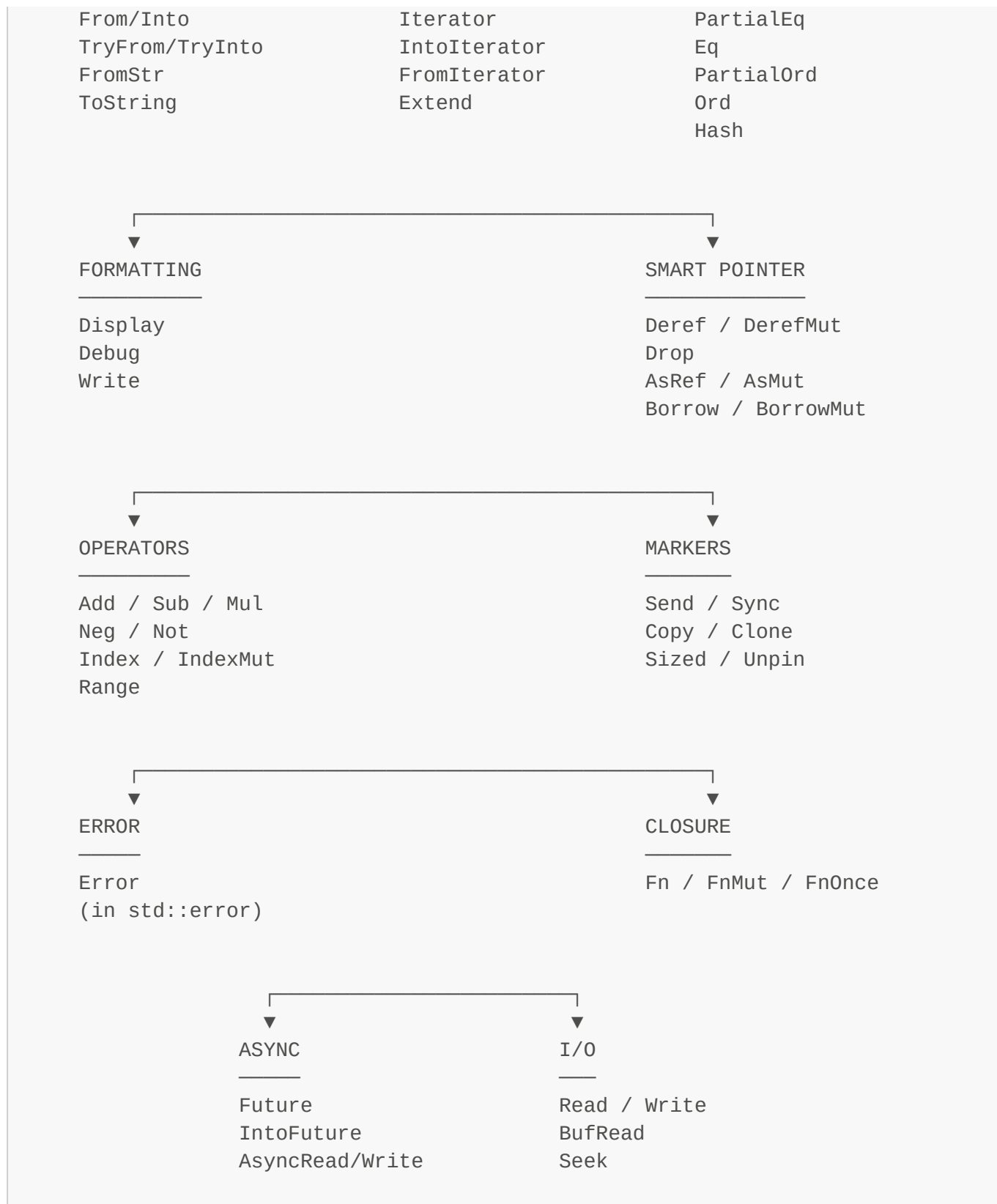
LUÔN impl From, KHÔNG impl Into

→ 1 dòng code, được cả 2 chiều

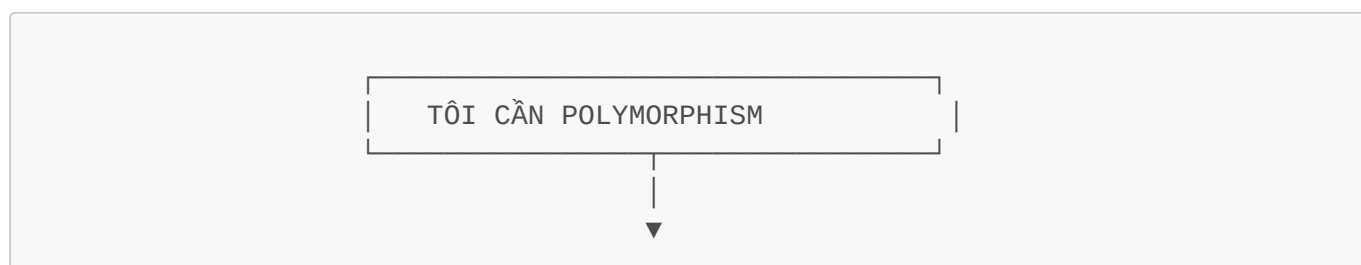
→ Lỗi compile dễ đọc hơn

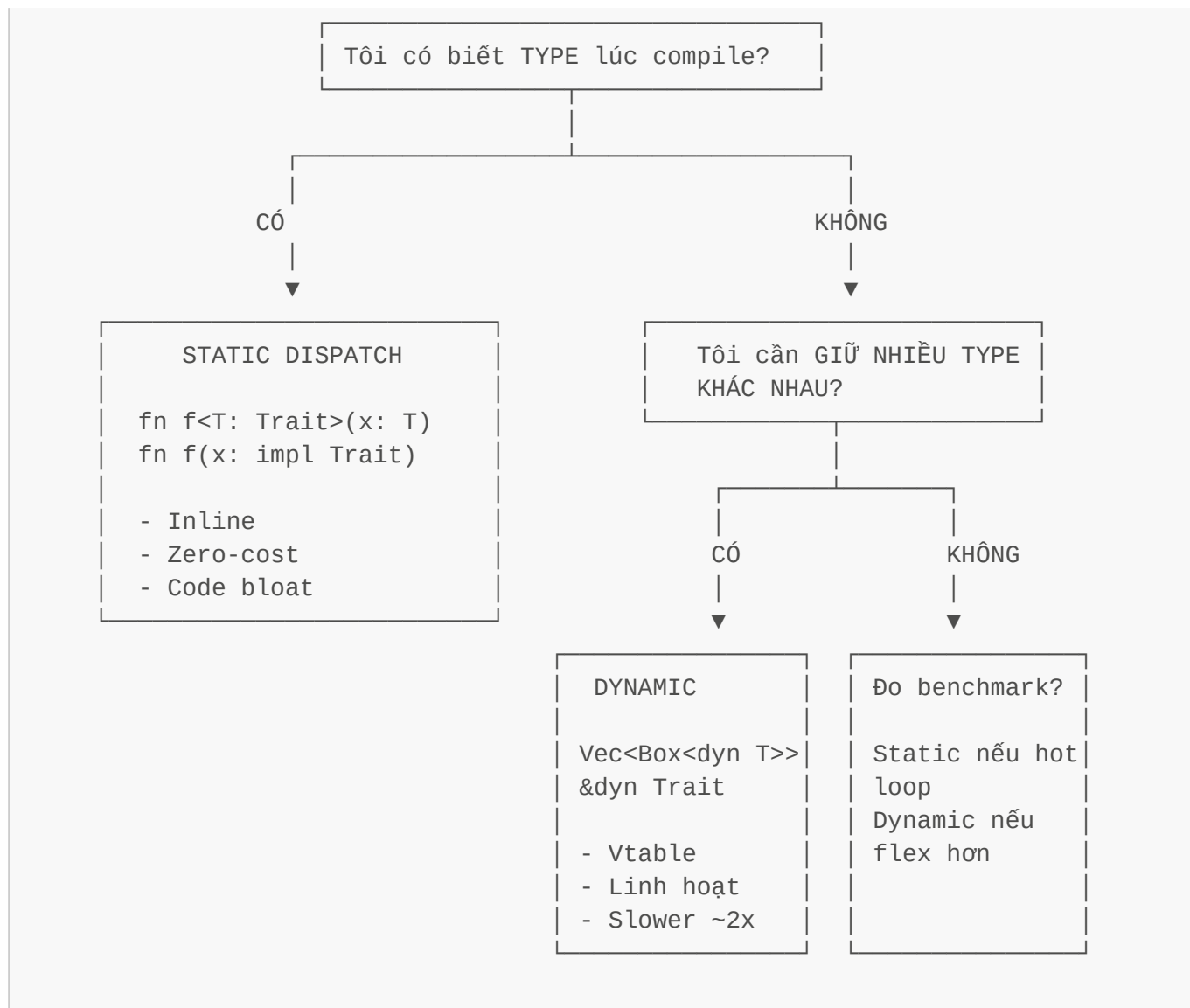
22. Bản đồ trait stdlib





23. Decision tree: static hay dynamic?

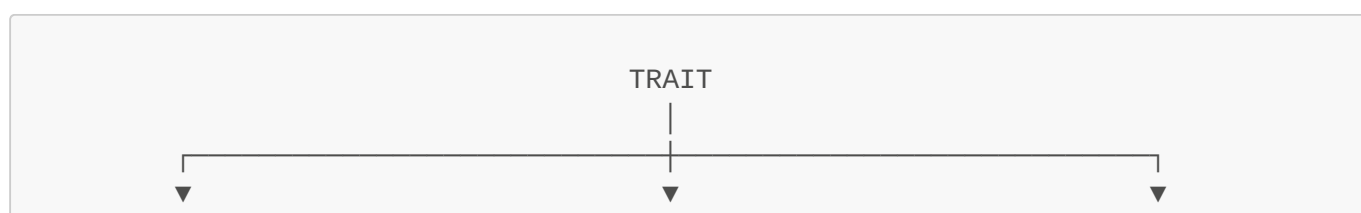


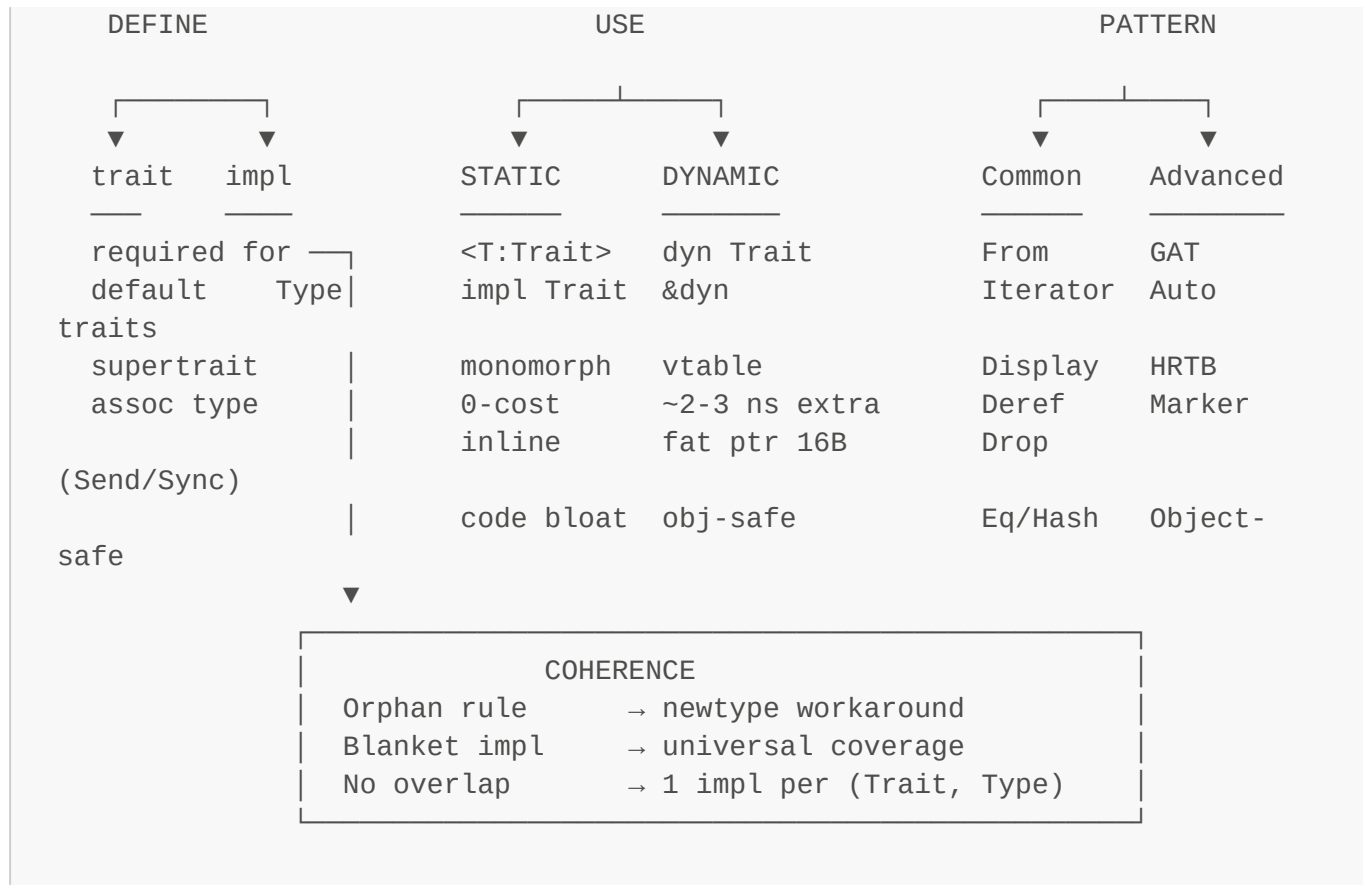


Use cases thực

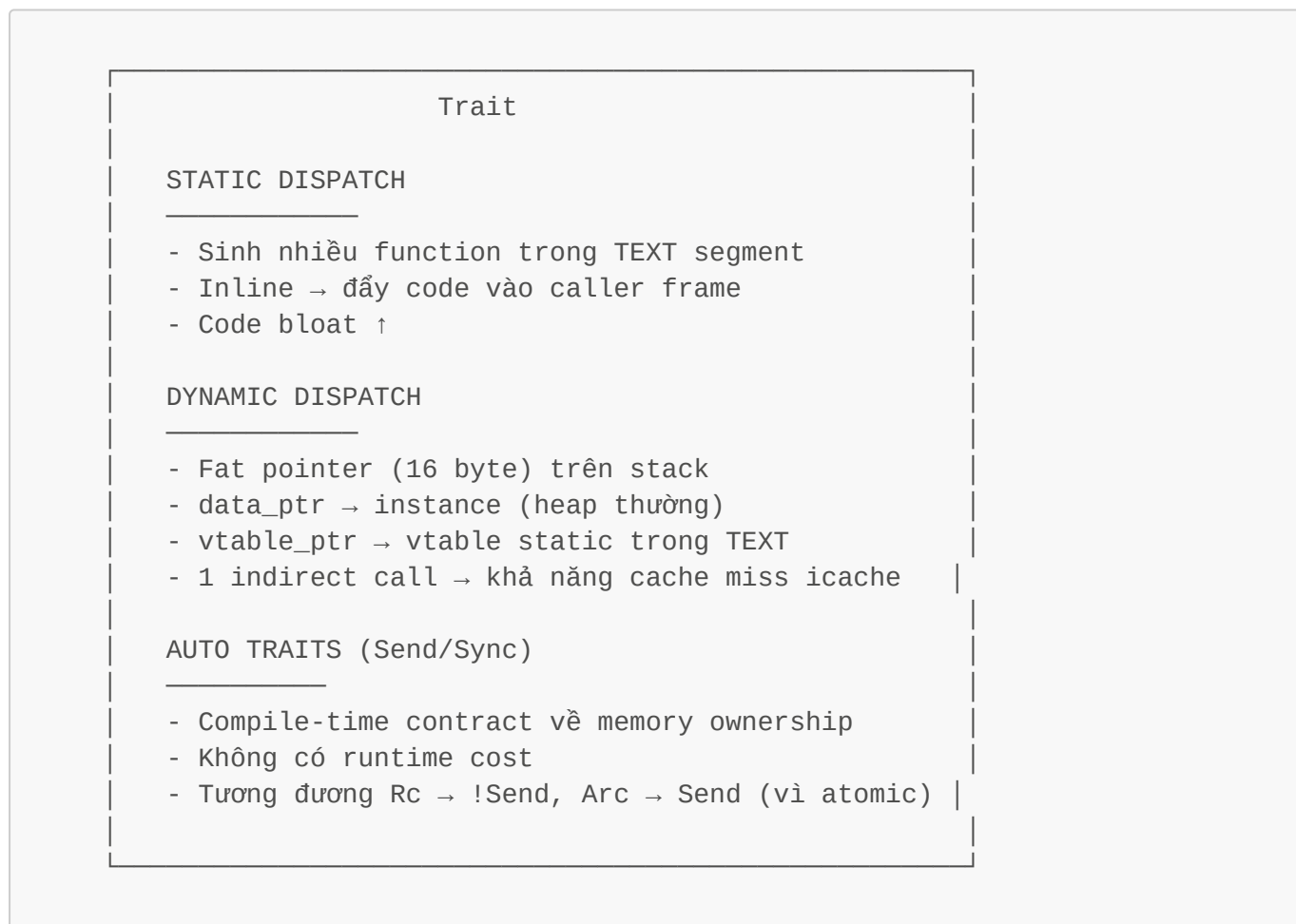
<u>STATIC dispatch dùng cho:</u>	<u>DYNAMIC dispatch dùng cho:</u>
- Iterator chain - Math/SIMD functions - Generic data structures - Hot loops - Small functions	- GUI widget tree - Plugin system - Heterogeneous collections - Error trait (<code>Box<dyn Error></code>) - Strategy pattern - Callback registry

24. Mind Map — Bản đồ tư duy tổng





Quan hệ Trait với Memory Model



Tổng kết — 10 ý cốt lõi

1. Trait = contract về CAPABILITY, không phải class
2. Type không inherit type, chỉ có trait (composition)
3. Static dispatch = monomorphization (compile-time)
→ nhanh, code lớn
4. Dynamic dispatch = vtable (runtime)
→ linh hoạt, code nhỏ, +2-3ns/call
5. dyn Trait = fat pointer (data + vtable), 16 byte
6. Object-safe trait mới dùng được dạng dyn
(no generic method, no return Self)
7. Orphan rule: Trait hoặc Type phải LOCAL
→ workaround: newtype pattern (zero-cost)
8. Associated type: 1 type, 1 impl, 1 Item
Generic param: 1 type, N impl, N output
9. Default method giúp trait scale
(Iterator: 1 required + 70 default)
10. Marker trait (Send/Sync/Copy) = auto-derived contract
compiler kiểm tra, zero runtime cost

Đọc song song [trait.md](#) (lý thuyết) để hiểu nguyên lý, file này để có hình ảnh trong đầu.

Chủ đề tiếp theo: **Generic** — xây trên trait bounds, hiểu thêm về monomorphization, where clauses, lifetimes trong generic, phantom types, const generics.